CS 498: Machine Perception
Final Project — Track 2: Complete 3D Scene Reconstruction

Fall 2026

Base credit	20 points
Work mode	Individual; AI assistance encouraged
Novel-idea bonus	Up to +2 for a substantive implemented and tested idea
Ranking bonus	+2 points for each eligible top-five Test entry
Due	December 9, 2026, 11:59 p.m. America/Chicago

1 What you will build

Reconstruct a *complete, textured, instance-separated 3D scene* from a posed RGB video. Recover each object's geometry, appearance, and world placement, including plausible backs, undersides, and occluded parts rather than only visible surface fragments. The assembled scene should avoid interpenetration. Evaluation uses static geometric collision checks: **no gravity simulation is required or scored**. Collision-free placement alone does not guarantee dynamic stability.

We value original ideas and thoughtful experiments. Explain what you tried and learned. A substantive implemented idea with real Val/Test evidence can earn the separate novel-idea bonus (Section 6).

You receive a runnable baseline, data, and a Dev/Val evaluator; **no TODO functions are required**. AI assistance is encouraged: verify suggestions and record decisions. Inference must be automatic, with no manual masks, object placement, mesh editing, or per-scene fixes.

Figure 1 illustrates the target representation. Complete geometry includes plausible unseen sides, not a guarantee that every hidden detail can be recovered from the input. Keep instance IDs consistent across the scene, isolated-object views, GLBs, and report.

2 Data, inputs, and coordinates

Supported commands are under `scripts/student/`; your algorithm lives in `methods/`; reusable settings are in `configs/`. Setup scripts, environment specifications, and data installation are in `scripts/setup/`, `env/`, and `data/`. Shared implementation is in `src/`, not another set of required tasks. `examples/` contains editable submission metadata and AI-record examples. Run commands from `handout/`.

Split	Scenes	GT access	Intended use
Dev	30	Available	Development, training, and controlled experiments
Val	10	Available	Local evaluation and model selection; no GT training
Test	20	Private	Frozen automatic inference; select one final version

Each scene has 192 frames at 512×512, 24 fps, for eight seconds. All data are provided; do not generate scenes. The three splits use separate base assets and variants. Download from the course dataset using the pinned checksum manifest, rather than fetching arbitrary repository files:

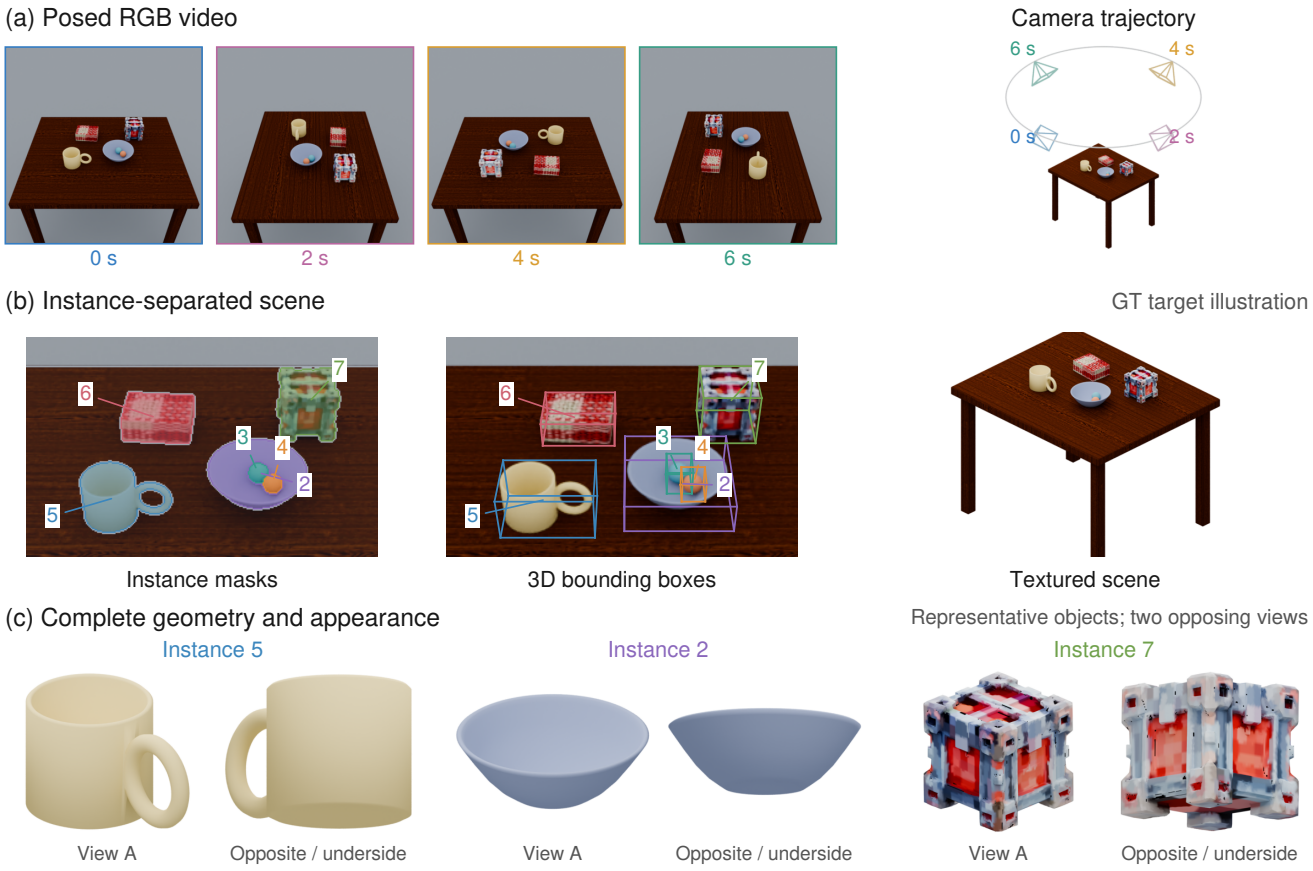


Figure 1: **Project inputs and target representation.** (a) Four RGB frames and their calibrated camera poses from public Dev scene 004; colors associate frames with poses. (b) GT instance masks, projected 3D boxes, and the assembled textured scene. (c) Opposing views of three representative GT objects, including underside views, show complete geometry and appearance rather than single-view surface fragments. All output-side panels use *GT to illustrate the goal*, not measured baseline predictions. Instance IDs, not category names, identify objects; the table is instance 1. Masks and boxes explain the representation and are not extra submission files.

```
# Run all commands from handout/.
bash scripts/setup/tools.sh
source env/tools/.venv/bin/activate
python -m pytest tests -q
python data/download_data.py --split dev
python data/download_data.py --split val
# Download Test when ready for final inference:
python data/download_data.py --split test
```

The tools installer requires Python 3 with `venv` support (Ubuntu: `python3-venv`). It provisions isolated Python 3.11.16 and pinned tools; it does not replace system Python or download baseline models.

The public location is the course Hugging Face dataset (`hongchi/cs498`, `final-track2/`). The downloader uses immutable archive URLs and SHA-256 verification. A local archive distribution can also be installed with `--from-local-dir=/path/to/archives`. Approximately 3.9 GB of public archives are supplied; budget space for both archives and extracted files, plus models, environments, and temporary reconstruction outputs.

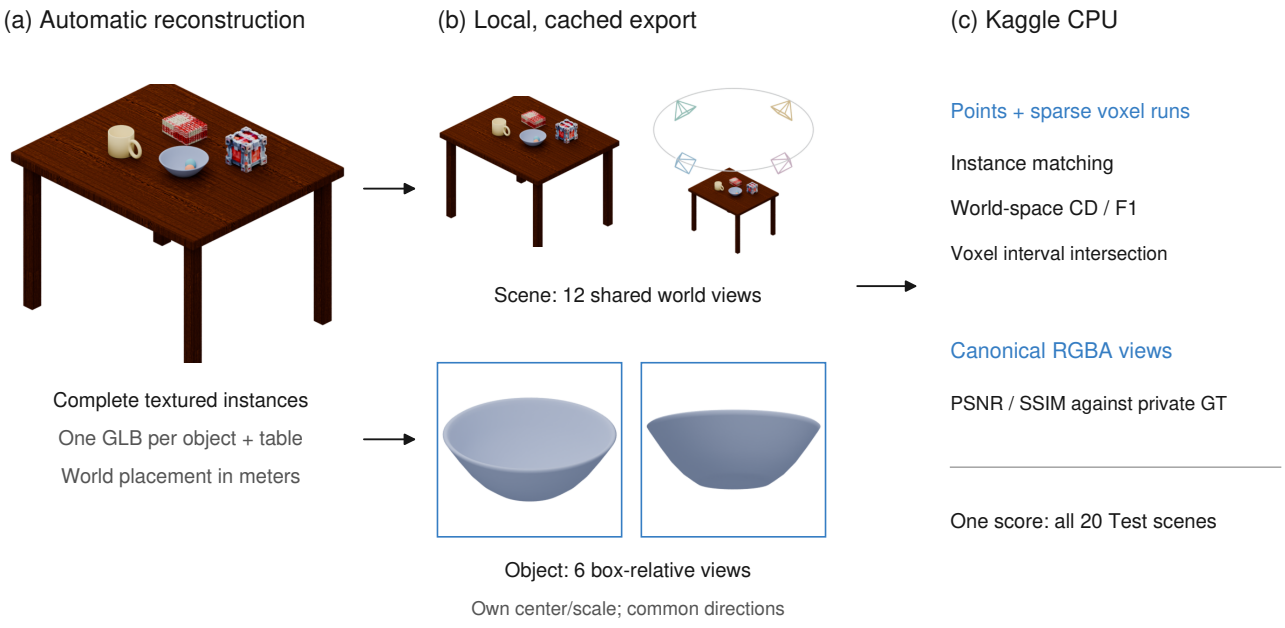


Figure 2: **One representation, automatic evaluation.** Reconstruct locally from RGB and calibration, then export the final meshes and canonical prediction views, surface samples, and sparse voxel runs locally. Scene cameras are fixed in the shared world frame; isolated-object cameras are centered and uniformly framed on each side's own box, using the same world-axis directions. Kaggle compares these predictions with private targets and computes geometry and collision checks on CPU. No student metric values, simulation trajectories, or private Test GT are needed for submission. Illustrative images use public Dev GT, not measured predictions.

```
data/scenes/<dev|val|test>/scene_XXX/
  rgb/frame_0000.png ... frame_0191.png
  cameras.npz
  scene.blend # evaluation GT: Dev/Val ONLY
  physics_eval.json # legacy metadata: unused; Dev/Val ONLY
```

Inference reads only RGB and the supplied calibration/poses, even when Dev/Val GT exists locally. No GT depth, masks, meshes, asset identities, or generation metadata may enter prediction. Dev GT can supervise training; Val GT is for measurement and model selection. Test GT is never distributed. Do not hardcode case IDs, filenames, or hidden asset lookups to manufacture predictions.

2.1 Coordinate checklist

Camera CV coordinates are x-right, y-down, z-forward. `camera_to_world_cv` maps camera points to the right-handed, Z-up dataset world in meters. `camera_to_world_blender` uses Blender's camera axes; these matrices are not interchangeable. `K` stores intrinsics; `frame_indices` determines temporal order. Use `track2_io.load_scene`, which loads input RGB and numeric calibration only.

Your algorithm returns meshes with world placement already applied. The shared exporter converts Z-up dataset coordinates to standard glTF Y-up coordinates: $(x, y, z)_{\text{dataset}} \mapsto (x, z, -y)_{\text{glTF}}$. Do not apply this conversion yourself when using `export_scene`; doing it twice rotates the scene incorrectly. Do not independently rescale or align objects to GT. Instance IDs are arbitrary positive integers, unique within a scene; they need not match GT IDs. **Category names are not evaluated.**

3 Run and understand the provided baseline

3.1 Environment and first run

The full baseline uses Linux, an NVIDIA CUDA GPU, Git, `uv`, Conda, and the system `ffmpeg` package (including `ffprobe`). Its two model runtimes intentionally use different Python/Torch versions: perception uses Python 3.11/Torch 2.5.1, reconstruction Python 3.10/Torch 2.7.1. Native reconstruction extensions need a CUDA toolkit and C++ compiler; use the provided setup scripts rather than merging the environments. An RTX 4080 with 16 GiB was used for the measured reference. Set `TORCH_CUDA_ARCH_LIST` appropriately for a different GPU before compiling. Budget at least 30 GiB scratch per active case, separately from persistent data/model/environment storage. Reconstruction weights occupy about 45 GiB; allow roughly 120 GiB or more in total for runtimes, models, builds, data, and an active case. This is not a download-time guarantee.

```
bash scripts/setup/perception.sh
bash scripts/setup/reconstruction.sh
python scripts/student/run_baseline.py --scene dev_scene_000 --check
python scripts/student/run_baseline.py --scene dev_scene_000
```

The output is `outputs/baseline/dev_scene_000/evaluation/mesh_manifest.json`. The default `infer` stage does not open a GT `scene.blend`; it works on the input-only Test package as well. Results are resumable. Use a new `--algorithm-name` whenever configuration or input changes; do not mix artifacts from different experiments.

3.2 What the pipeline does

1. **Predict geometry.** Pi3X, the official engineering extension of π^3 [1, 2], estimates RGB-D geometry using images and known camera calibration. GT depth is not an input.
2. **Discover and associate instances.** Florence-2 [3] proposes regions, SAM 2 [4] estimates masks, and multi-view 3D fusion groups observations. The supplied baseline retains tracks supported in at least 25% of distinct input frames and applies class-agnostic 3D bbox NMS at IoU 0.65. Model-generated labels can help its internal association, but label correctness is not scored.
3. **Complete geometry and appearance.** FIRE3D [5] uses fused predicted instances for geometric completion and PBR appearance. It is not the perception module in this baseline. Its conditioning uses DINOv3 image features [6] upsampled by AnyUp [7]; its shape and material decoding uses the TRELLIS.2 SC-VAE decoders [8]. The reference uses uniform canonical scaling, 12 flow steps, and remesh band 2.0; these are baseline settings, not restrictions on another method.
4. **Export.** Every object and the table become separate textured GLBs in world coordinates, with a common manifest and calibration metadata.

Measured original end-to-end runs were approximately 7–11 minutes per scene on the reference GPU; hardware and scene complexity matter. Native remesh/UV operations are not guaranteed bitwise repeatable. Preserve the exact GLBs evaluated in each experiment rather than assuming a seed reproduces them.

4 Evaluate, visualize, and diagnose

Rendering runs locally with Blender 4.2.0 [9]; numerical scoring uses the same CPU implementation as Kaggle. CPU rendering is supported by default. A configured NVIDIA OptiX device may accelerate it with `--device=gpu`; keep all canonical camera, lighting, resolution, and sample-count settings unchanged.

```
bash scripts/setup/evaluation.sh
export BLENDER_EXECUTABLE="$PWD/env/blender/blender-4.2.0-linux-x64/blender"
python scripts/student/evaluate.py --manifest \
  outputs/baseline/dev_scene_000/evaluation/mesh_manifest.json
```

Open `evaluation/reports/evaluation/default/index.html`. Its paired views show GT on the left and predictions on the right. Retain `metrics.json`, `matching.json`, `visualizations.json`, and linked images. Inspect missing/duplicate/merged instances, world placement, thin or occluded geometry, complete object appearance, and collision witnesses rather than only the total. Use a new `--run-name` for a changed method or result.

4.1 Metric definitions

Perception. One-to-one, class-agnostic matching uses world-space AABB IoU at threshold 0.25, maximizing eligible match count before total IoU. Foreground instance F1 penalizes missing objects and unmatched extras. The support table participates in matching but is excluded from foreground counts; **category names are not required**.

Geometry. Sample complete surfaces uniformly by triangle area: 100,000 points for the scene union, including the table but not the floor, and 20,000 points per object. Sampling is performed and cached locally, with public fixed seeds independent of GT instance IDs; Kaggle only compares the submitted and precomputed GT points. Symmetric *unsquared* Chamfer Distance is in meters; surface F1 uses 1 cm. The normalized geometry score is

$$G = \tfrac{1}{2}\left(\exp[-\mathrm{CD}/(0.05d)] + F_{1\mathrm{cm}}\right),$$

where d is the GT bounding-box diagonal. The object group is foreground matching recall times the mean score over *all matched instances*, including a matched support table; an empty match list earns zero. Foreground omissions reduce this coverage multiplier. Support omissions instead affect scene geometry/appearance and make collision credit zero; the object group alone is not a support-completeness criterion. There is no ICP, per-object rescaling, or GT pose correction for these world-space metrics.

Appearance. Compare 12 scene views using identical fixed world cameras derived only from the public input calibration. Do not frame the scene from predicted or private GT bounds. For each matched object, use six orthographic world-axis directions $(\pm X, \pm Y, \pm Z)$, independently centered on that side's own AABB and uniformly framed by its longest edge. There is no independent rotation or nonuniform stretching. Thus object appearance measures shape/material agreement separately from size and placement, which remain measured by world-space geometry/matching.

Canonical renders use 512×512 RGBA PNGs, 32 Cycles samples, neutral lighting, and the Standard view transform. Composite on white and average normalized PSNR (40 dB ceiling) and SSIM [10], with equal full-image and foreground-union contributions. Object appearance also carries the foreground recall multiplier. Do not change the renderer or submit edited or independently generated metric images.

Collision-free placement. This is an approximate static test, not simulation. The local exporter builds a **5 mm world-anchored sparse voxel grid**. Cell (i, j, k) has center $(i + 0.5, j + 0.5, k + 0.5) \times 0.005$ meters. Preserve separate closed concave components; open 3D components use a convex fallback. An occupied cell contains an interior grid center or a quantized sampled surface point. Each component's core is its interior eroded by a $3 \times 3 \times 3$ cell neighborhood. Instances collide if either one's core intersects the other's occupied cells. Planar pieces contribute sampled surface cells but no invented volume.

Kaggle only intersects sparse integer column intervals; it never voxelizes or examines triangles. Core erosion avoids penalizing ordinary contact, but may miss sub-grid or thin penetrations. This is not exact mesh collision, support testing, or a physical-stability guarantee. Floor penetration is checked separately from the original minimum Z, with a 2 mm tolerance. Invalid or excessively fragmented collision solids earn no collision credit. If any predicted instance cannot be voxelized, that scene's C is zero.

Define

$$C = \frac{\#\{\text{GT foreground instances matched to collision-free predictions}\}}{\#\{\text{all GT foreground instances}\}}.$$

The table must also be matched to receive collision credit. Missing objects earn no credit; extra objects participate in collision tests but earn no extra credit. Floating objects can still be collision-free:

this metric does not certify support or stability, and world placement remains graded separately. A local 600-second voxelization deadline or collision budget failure gives **zero for that scene's collision component only**; other groups remain intact. Non-timeout infrastructure errors require recovery, not invented zero scores.

4.2 Aggregate references and split summaries

Reference means cover all 30 Dev, 10 Val, and 20 Test scenes. Components in the table are normalized to 0–100, not raw PSNR/CD. These are contextual measurements, not an attainment requirement for the eight evidence points. Evaluate the exact exported GLBs: 20 MB texture packing may affect appearance. Machine-readable means are in `configs/baseline_reference.json`.

Metric	Dev (30)	Val (10)	Test (20)
Benchmark total /100	45.45	33.73	25.21
Instance F1	66.27	43.21	22.55
Scene geometry	59.01	57.46	52.99
Object geometry	38.27	19.54	7.55
Scene appearance	52.04	49.03	49.29
Object appearance	32.16	15.59	7.99
Collision-free recall	1.11	0.00	0.00

After evaluating all Val scenes:

```
python scripts/student/summarize_scores.py \
  --predictions outputs/my_method --split val
```

Average all ten cases; missing/failed cases contribute zero. Keep the per-case reports linked to the same final configuration/model and GLBs.

5 Further improve with your own algorithm

Explore new ideas; you need not retain the baseline architecture or even use a neural method. Novelty is relative to the supplied baseline, not a claim of being first in the literature. Possible directions include better proposals, multi-view association, depth/pose consistency, object-centric or joint scene completion, appearance estimation, automatic support estimation, and collision-aware optimization. Explain the motivation and measured effects of your choices; these are suggestions, not required methods or TODO tasks.

5.1 The open-method interface

Implement a function in `methods/my_method.py`:

```
def predict(scene):
    image = scene.rgb(0) # uint8 RGB; all frames are accessible
    cameras = scene.calibration
    # Run your fully automatic algorithm here.
    # Each mesh: trimesh.Trimesh, dataset world, Z-up, meters.
    return {1: table_mesh, 2: object_mesh, 3: another_object_mesh}
```

```
python scripts/student/run_method.py --method methods.my_method:predict --scene dev_scene_000
python scripts/student/evaluate.py --manifest \
  outputs/my_method/dev_scene_000/evaluation/mesh_manifest.json
```

`methods/interface_example.py` is a runnable one-box format demonstration, *not* a meaningful reconstruction baseline. Use it to test IO, then replace its prediction logic. You may implement your method outside this adapter if you export exactly the same files. Include a separate predicted table mesh; do not fuse all objects into one instance. ID values do not encode category.

`track2_io.export_scene` handles coordinates, manifests, and embedded PBR textures. It preserves existing UV textures; vertex/face colors can be converted to a simple atlas. Constant metallic/roughness textures are provided when absent, so a non-generative method can satisfy the format. This is only format conversion: it does not recover missing appearance or guarantee quality.

If modifying the supplied baseline, create an experimental copy/configuration and a new output name. Its frozen-profile and protocol assertions protect reference reproduction, not student freedom: replace those checks deliberately when changing its algorithm, or use the unrestricted interface above. Disclose pretrained models, training data, dependencies, and licenses. Your method and configuration must be the same across cases.

5.2 Make a defensible improvement claim

Start with one Dev scene and inspect intermediate failures. Change one factor at a time, then evaluate all Val scenes. For each claimed improvement, include a controlled comparison keeping other choices fixed. Better visual appearance can harm geometry; more instances can introduce interpenetration; overly smooth meshes can remove thin supports. Explain those trade-offs rather than selecting only the favorable metric. Freeze the final method before Test inference. During development you may try many variants, but submit **one final version**, selected by the highest mean benchmark score across **all ten Val scenes**. Use one method/configuration across cases; do not combine the best case or metric from different versions. The report's final Val table, code/model snapshot, and Test outputs must describe that same frozen version. Baseline and controlled comparisons may support your analysis; they are not additional final entries or additional Test attempts.

6 Grading: 20 points plus bonuses

Component	Scoring evidence	Points
Results and visuals	Three selected scenes from each of Dev, Val, and Test; traceable evidence	8
Perception	Class-agnostic instance F1	1.5
Reconstruction	Scene/object geometry and appearance	1.5
Collision-free	Coverage-aware collision-free foreground recall	1.0
Report	Ideas, experiments, reproducibility, AI collaboration	8
Base total	Excludes both bonuses below	20
Ranking bonus	Each eligible top-five Test entry; reproduction required	+2
Novel-idea bonus	Substantive implemented idea, controlled Val evaluation and official Test results	Up to +2
Maximum with bonuses	Both bonuses are independent	24

6.1 Results and visual evidence: eight points

Show **three distinct scenes from each split**: nine selected scenes from the same final method. Name scene and instance IDs and link full dumps. There is **no baseline-score threshold** for these points. Each of the following four evidence groups is worth two points:

1. **Instances.** Dev/Val: matching results and ID-labeled boxes/views, including misses and extras. Test: prediction-only instance views; no GT matching is requested.
2. **Scene.** Dev/Val: world-space geometry/error evidence and paired GT/prediction appearance. Test: multiple prediction-only scene views.
3. **Objects.** Dev/Val: per-object geometry and appearance comparisons, with unmatched objects retained in the dump. Test: isolated predicted-object views labeled with their submitted IDs. Show representative objects in the report and retain complete matched-object evidence alongside it.
4. **Collision-free placement.** Dev/Val: collision status, reported proxy/fallback information, and marked interpenetration witnesses or an explicit clean result. Test: show prediction-only contact/-

placement views; official collision evidence is verified during final evaluation without giving students private GT.

For each group award $2n/9$, where n is the number of verified selected scenes. Duplicate/missing IDs do not count. Honest unmatched-object or timeout evidence is creditable; fabricated success or an absent dump is not. No post-evaluation report resubmission is required.

For selected Dev/Val scenes preserve the complete generated report folder. For Test, use the GT-free preview helper:

```
python scripts/student/preview_outputs.py --manifest \
  outputs/my_method/test_scene_000/evaluation/mesh_manifest.json
```

Keep its ID-labeled scene/object montages under the result archive's `evaluation/test/` tree, not the minimal mesh ZIP. These are qualitative previews, not local Test scores. Canonical Kaggle renders are generated separately by `export_kaggle.py`.

6.2 Exactly how numeric credit is calculated

The four measured-performance points are perception 1.5, reconstruction 1.5, and collision-free 1.0. First compute each normalized metric's mean over *all scenes in a split*. Val and Test receive equal weight despite their different sizes. The absolute 75% threshold applies independently to each split mean, not every case and not a fraction of the reference score.

For $q \in [0, 1]$, let $f(q) = \min(1, q/0.75)$:

- Perception: $0.75f(q_{\text{Val}}) + 0.75f(q_{\text{Test}})$.
- Collision-free: $0.5f(q_{\text{Val}}) + 0.5f(q_{\text{Test}})$.
- Each of scene geometry, object geometry, scene appearance, and object appearance: $\frac{3}{16}f(q_{\text{Val}}) + \frac{3}{16}f(q_{\text{Test}})$.

Thus 0.50 earns two thirds of that split's available credit. Strong performance on one reconstruction metric does not hide a weak one.

The *separate* Kaggle ranking score is the mean over all 20 Test cases of

$$100(0.20F_{\text{inst}} + 0.20G_s + 0.20G_o + 0.15A_s + 0.15A_o + 0.10C).$$

All components lie in $[0, 1]$. No second Test subset is used. The ranking score is not the entire course grade.

6.3 Report, eligibility, and bonuses

Report credit has four two-point criteria: (1) original ideas, sound method, rationale and related work; (2) quantitative experiments, controlled comparisons, visual analysis and limitations; (3) exact reproducibility and artifact identity; (4) meaningful AI–human collaboration, decisions and verification. Partial credit uses quarter-point increments. Clear hypotheses and informative unsuccessful experiments are valued; routine grading is based on the report and evidence, not line-by-line code inspection.

Submit **one final method**, chosen by the highest mean benchmark over all ten Val scenes. Final Val results, code/model/configuration, Test meshes and Kaggle CSV must describe that same frozen method. Comparisons are supporting experiments, not per-case/per-metric oracle selection or extra final entries.

The top five eligible, reproducible official Test entries earn **+2 each**. Results remain subject to artifact/code/model and policy checks. A substantive new idea in the same final method earns **up to +2**, independently of ranking: 0 without implementation, full-Val measurements and official Test results; 1 with substantive implemented/tested work but limited controls/analysis; 2 with reproducible controlled evaluation and clear interpretation. A gain on every metric is not required. A proposal, run-name, presentation or compute-budget change alone is insufficient. Both bonuses use the existing final deliverables; maximum total is 24.

If Test is entirely absent, assess numeric work from Val and evidence over the six Dev/Val scenes ($2n/6$ per group), then halve the *entire* base once: maximum 10/20, neither bonus eligible. A partial

or oversized Test bundle must be repaired before submission. A collision timeout zeroes collision only; evaluation infrastructure failures remain pending until recovered.

7 Kaggle challenge and final submission

7.1 Practice the submission format locally on Val

Use public Val GT to understand and test the submission contract before any Kaggle upload. Start with one completed Val reconstruction:

```
python scripts/student/export_kaggle.py \
  --predictions outputs/my_method --split val --scenes val_scene_000 \
  --output outputs/val_one.csv --device cpu
python scripts/student/check_kaggle_submission.py \
  --csv outputs/val_one.csv --scenes val_scene_000 \
  --predictions outputs/my_method --output outputs/val_one_check.json
```

The checker validates the CSV, bounded images/arrays/voxel runs, GLB source identity and the strict 20 MB budget. It computes all groups with the **same numerical scorer as Kaggle**, using public Val GT locally, and writes a JSON receipt. Its `submission_format` inventory explains the actual IDs, point/bounds shapes, image counts, sparse voxel-run counts and preprocessing status without requiring you to edit encoded chunks. No Test GT or online submission is used. This format rehearsal does not replace the required paired Dev/Val visual reports generated by `evaluate.py` and `summarize_scores.py`.

Next, omit `--scenes` in both commands and choose new filenames to check all ten Val scenes (20,480 rows). The one-scene diagnostic has 2,048 rows. For final Test export use `--split=test` and all 20 cases (40,960 rows). Only the split/scene inventory changes, not the representation or metric. Do not rename Val IDs into Test IDs or upload Val results to the competition. Initial rendering is local and potentially slow; unchanged views, samples and voxels are cached under `outputs/.evaluation_cache/`. Completed CSVs and check receipts are immutable; choose new names for changed results.

7.2 Export the final Test results

Run the frozen automatic method on all 20 Test inputs:

```
outputs/my_method/test_scene_000/evaluation/
  mesh_manifest.json
  meshes/obj_001.glb ...
  inputs/cameras.npz
  inputs/source.json
# Repeat through test_scene_019.
```

Use embedded textures and baked world placement. **All GLBs in a scene together must be strictly below 20,000,000 bytes**, including the table. Automatic UV/texture packing preserves geometry, IDs and world placement; re-evaluate after packing because appearance may change. The adapter permits at most 64 meshes, IDs in $[1, 32767]$, and combined scene totals of two million vertices and two million triangles. The portable scene record has an additional 80 MB uncompressed transport bound. Run the validators before uploading; do not omit difficult objects to fit.

```
python scripts/student/validate_submission.py \
  --predictions outputs/my_method --split test
python scripts/student/export_kaggle.py \
  --predictions outputs/my_method --split test \
  --output outputs/submission.csv --device cpu
```

Rendering, surface sampling, and voxelization are local, cached, and need no Test GT. GT views and samples are precomputed before competition setup. Use `--device=gpu` only with configured OptiX

hardware. Wait for the exporter to print its final CSV path: a `.partial` file is not a completed submission. Keep the exact final GLBs.

7.3 Submit the CSV to Kaggle

First open the course invitation with your individual Kaggle account. The restricted-access check uses an `illinois.edu` email address; follow Kaggle's verification prompts. A private-page/404 response before you join is expected. After joining, open `https://www.kaggle.com/competitions/cs498-fall-2026-track2-reconstruction`, accept the applicable rules, and upload the generated plain `submission.csv`. No course API token is required; never include credentials in a submission. Only this CSV goes to Kaggle; do not upload the course ZIP there.

The columns are `Id,Prediction`. Each scene has 2,048 fixed rows, with at most 65,536 characters per chunk; `-` denotes padding:

```
Id,Prediction
test_scene_000__0000,<generated chunk>
test_scene_000__0001,<generated chunk or ->
...
test_scene_019__2047,-
```

All 40,960 rows are required in sample order. The exporter handles this format; **do not hand-edit chunks**. Fixed slots accommodate different geometry/ image sizes without revealing target sizes. Kaggle reassembles scenes and computes matching, CD/F1, appearance and collision on CPU against private GT; it does not trust student-reported scalar metrics. The displayed score averages the same full 20-case set for ranking, without a separate hidden subset. Check the submission status, not only upload success. Queue and execution time can vary; format errors must be fixed before a valid score.

CSV compression/encoding is not encryption or an electronic signature. Images must come from the exact submitted GLBs and unchanged canonical renderer. Hashes are reproducibility receipts, not proof of image production. Point samples and voxel grids must also be produced from those GLBs by the unchanged exporter. Final code/model/artifact review may request local reproduction; production Kaggle scoring never renders or parses meshes. Never fabricate representations, copy private targets, exploit evaluator errors, or manually repair Test scenes.

Submission policy. Kaggle permits at most **one submission per day (UTC)**. Later-day submissions are allowed, but select **one final complete version** using the highest mean benchmark across all ten Val scenes, not Test feedback. Select its matching final entry on Kaggle and include the exact submission ID and CSV in the HF result archive referenced by the course ZIP. Do not combine different methods per case or per metric. Local Dev/Val and format checks are unlimited. Infrastructure recovery resumes the same immutable artifacts.

7.4 Prepare the final results

Package the minimal mesh archive:

```
python scripts/student/package_submission.py \
  --predictions outputs/my_method --output outputs/test_results.zip
```

It contains only all-20-case GLBs, manifests, calibration/source metadata and a checksum receipt, not models, reports, videos, GT, environments or caches.

Prepare the complete local `submission/` staging tree below. The next step splits it into a large HF result archive and a small course-platform ZIP:

```
submission/
  report.pdf
  README.md
  code/ # README, final_config, dependencies, final source
  models/manifest.json # paths/HTTPS URLs and SHA-256
  models/... # optional local weights/licenses; moved to HF
  ai_collaboration.jsonl
  test_results.zip
  kaggle_submission.csv # exact uploaded CSV, renamed only
  kaggle_receipt.json # submission ID and CSV SHA-256
  evaluation/val_summary.json
  evaluation/evidence_selection.json
  evaluation/dev/<selected-scene>/evaluation/reports/evaluation/default/...
  evaluation/val/<scene>/evaluation/reports/evaluation/default/...
  evaluation/test/<selected-scene>/prediction_preview/...
```

Include all available ten-case Val metrics and full evidence for three selected Dev and three Val scenes. Test report evidence is prediction-only; you do not need private Test GT or a post-evaluation report revision. Use `examples/submission_selection.json` and `examples/model_manifest.json`. Model declarations name every needed checkpoint using either an included file or a stable HTTPS URL and its file SHA-256; use `required:false` with an empty list only for methods that need no weights. Include actual AI decisions, not the sample log entries. The `code/` folder must include `README.md`, `final_config.json`, your final source, and `requirements.txt` or an equivalent dependency declaration. Do not package credentials, data, caches, environments, Git history, or expiring signed links. Every artifact must identify the same final method.

Record the actual Kaggle submission ID (from its details page) and CSV hash:

```
python scripts/student/kaggle_receipt.py \
  --csv outputs/submission.csv --submission-id YOUR_SUBMISSION_ID \
  --output submission/kaggle_receipt.json
```

Copy the same CSV to `submission/kaggle_submission.csv` without changing its bytes. The receipt is a traceability record; official status/score are verified from Kaggle, not accepted from a self-entered claim.

7.5 Host large results in your own HF dataset

```
python scripts/student/project_submission.py --submission-dir submission \
  --netid YOUR_NETID --prepare-hf-results --output outputs/results.zip
```

Create a dataset repository in your own account at `https://huggingface.co/new-dataset`. In Files and versions, use Add file to upload this archive as exactly `results.zip`. It contains the `evaluation/` tree, Test mesh ZIP, exact Kaggle CSV/receipt, optional local weights and a generated inventory. Do not upload your working folder, source, report, AI record, credentials, course data, environments or caches. Redistribute weights only with permission; otherwise declare the original provider's stable pinned URL and file SHA-256 in the model manifest.

Keep the repository **private during development**, then make it **public and ungated at final course hand-in**. The exact submitted revision must be downloadable without login. A private URL alone is not a completed hand-in; never submit tokens. Verify anonymous access before the deadline and retain the exact files through grading and appeals. Public predictions can be copied by competitors, so do not publish unfinished results during the competition. Passing off another participant's outputs as your own is forbidden.

Copy the **full 40-character commit hash** from the upload commit page. Do not use `main`, a tag, a short hash or an expiring URL. The report must state the dataset ID, full commit, `results.zip` SHA-256 and actual Kaggle submission ID, matching the generated pointer in the course ZIP.

```
python scripts/student/project_submission.py --submission-dir submission \
  --netid YOUR_NETID --output outputs/YOUR_NETID_track2.zip \
  --hf-results-zip outputs/results.zip --hf-repo YOUR_HF_USER/YOUR_REPO \
  --hf-revision YOUR_FULL_40_CHARACTER_COMMIT
```

Only the small ZIP below goes to the course system, without an enclosing folder:

```
report.pdf
README.md
code/... # actual source, final config and dependencies
models/manifest.json # URLs or paths restored from HF results
ai_collaboration.jsonl
results_location.json # generated HF dataset/commit/bytes/SHA-256
submission_manifest.json # generated inventory
```

Code must still be submitted. An HF/GitHub code link does not replace the source snapshot. Local weights move to HF with the result archive, while their model declaration remains in the course ZIP. The packer checks core layout, Val means, immutable input calibration, all 20 Test scenes, mesh budgets, and the decoded CSV's instance IDs, GLB hashes and byte counts against the inner mesh archive; it adds inventories and a fixed-revision pointer. It does not upload to HF or prove remote accessibility: download the pinned revision's `results.zip` to a fresh directory and compare its SHA-256 before hand-in. These are consistency checks, not proof of prediction authenticity. If outputs change, make a new archive/commit and regenerate the course ZIP and report pointer. Never silently replace the submitted version. The results ZIP safety bound is 8 GB, not a per-scene mesh allowance. The grader checks the pinned archive's size/hash and restores the canonical tree, without executing code. Access failures require recovery of the exact submitted artifact, not an automatic numeric zero or a new method. Keep both archive identities. Missing visual evidence produces review warnings; core-format errors stop packaging.

If intentionally omitting Test entirely, omit the Test mesh ZIP and Kaggle files, set the selection's `test` list to `[]`, and use `--without-test`; the one whole-base halving and no-bonus rules apply. A nineteen-scene bundle is not an accepted twenty-scene submission.

Build the report. Edit `report.tex` and keep `references.bib` and your figures beside it. For a browser-based route, upload these files to Overleaf, set `report.tex` as the main document, and use pdfLaTeX. Recompile and download the result as `report.pdf`. On Ubuntu, install `texlive-latex-extra` and `texlive-fonts-recommended`, then run:

```
pdflatex -interaction=nonstopmode -halt-on-error report.tex
# If you uncomment the bibliography commands and use citations:
bibtex report
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

With no bibliography, only the first command is needed. Inspect the final PDF, remove template instructions, and place it at `submission/report.pdf`.

8 AI collaboration and final checklist

You are encouraged to use AI for literature exploration, algorithm design, coding, debugging, and experiment interpretation. Record meaningful tasks, important suggestions, what you accepted/rejected and why, verification, and failures or fixes. The template `examples/ai_collaboration.jsonl` is an example format, not a requirement to preserve its sample entries. A concise record of real decisions is more useful than an unfiltered chat dump or token count.

You remain responsible for understanding the final method, checking generated code, citing sources, respecting dataset boundaries, and reporting results truthfully. Do not send credentials or private data to external tools. AI assistance during development does not allow human intervention during per-case inference or hand editing the final outputs.

Before submitting, verify: your input is RGB/calibration only; all cases run with one automatic configuration; objects and table are separate; coordinates and scale are correct; all ten Val cases contribute to the mean; Test output contains all twenty cases; every scene's GLBs total below 20 MB; the one outer ZIP contains report, source, model declarations, AI record and the pinned HF pointer; the readable HF archive contains evidence, the inner Test ZIP and exact Kaggle CSV/receipt; local validation passes and both archive identities are recorded; the report explains controlled improvements, failure cases, and your AI-assisted decisions.

References

[1] Yifan Wang, Jianjun Zhou, Haoyi Zhu, Wenzheng Chang, Yang Zhou, Zizun Li, Junyi Chen, Jiangmiao Pang, Chunhua Shen, and Tong He. π^3: Permutation-equivariant visual geometry learning. *arXiv preprint arXiv:2507.13347*, 2025. URL `https://arxiv.org/abs/2507.13347`.

[2] Pi3 Contributors. Pi3X: Engineering update to π^3. Official code and model release, December 28, 2025. URL `https://github.com/yyfz/Pi3`. Software release, not a separate research paper.

[3] Bin Xiao, Haiping Wu, Weijian Xu, Xiyang Dai, Houdong Hu, Yumao Lu, Michael Zeng, Ce Liu, and Lu Yuan. Florence-2: Advancing a unified representation for a variety of vision tasks. *arXiv preprint arXiv:2311.06242*, 2023. URL `https://arxiv.org/abs/2311.06242`.

[4] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024. URL `https://arxiv.org/abs/2408.00714`.

[5] Hongchi Xia, Tianhang Cheng, Wei-Chiu Ma, and Shenlong Wang. FIRE3D: Feed-forward interactive 3D scene reconstruction within a minute. *arXiv preprint arXiv:2609.08848*, 2026. URL `https://arxiv.org/abs/2609.08848`.

[6] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. URL `https://arxiv.org/abs/2508.10104`.

[7] Thomas Wimmer, Prune Truong, Marie-Julie Rakotosaona, Michael Oechsle, Federico Tombari, Bernt Schiele, and Jan Eric Lenssen. AnyUp: Universal feature upsampling. In *International Conference on Learning Representations*, 2026. URL `https://arxiv.org/abs/2510.12764`.

[8] Jianfeng Xiang, Xiaoxue Chen, Sicheng Xu, Ruicheng Wang, Zelong Lv, Yu Deng, Hongyuan Zhu, Yue Dong, Hao Zhao, Nicholas Jing Yuan, and Jiaolong Yang. Native and compact structured latents for 3D generation. *arXiv preprint arXiv:2512.14692*, 2025. URL `https://arxiv.org/abs/2512.14692`. TRELLIS.2.

[9] Blender Foundation. Blender 4.2 LTS. Software release, 2024. URL `https://www.blender.org/download/releases/4-2/`.

[10] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004. doi: 10.1109/TIP.2003.819861. URL `https://www.cns.nyu.edu/~lcv/ssim/`.